CC–13 : Artificial Intelligence (Unit–3)

Knowledge representation: Humans are best at understanding, reasoning, and interpreting knowledge. Human knows things, which is knowledge and as per their knowledge they perform various actions in the real world. But how machines do all these things comes under knowledge representation and reasoning.

Following are the various **types of knowledge**:

1. **Declarative Knowledge –** It includes concepts, facts, and objects.
2. **Procedural Knowledge –** It includes rules, strategies, procedures, agendas, etc.
3. **Meta-knowledge –** Knowledge about the other types of knowledge is called Meta-knowledge.
4. **Heuristic knowledge –** Heuristic knowledge are the experiences in a field or subject which are good to work but not guaranteed.
5. **Structural knowledge –** It describes the relationship between concepts or objects.
6. **Knowledge-Base:** A knowledge base is an organized collection of facts about the system's domain.

Introduction to First Order Predicate Logic: Propositional logic (PL) is the simplest form of logic where all the statements are made by propositions. **It is a technique of knowledge representation in logical and mathematical form.** A proposition is a declarative statement which is either true or false e.g.

a) It is Sunday.
b) The Sun rises from West (False proposition)
c) 3+3= 7(False proposition)
d) 5 is a prime number.

In propositional logic, we can only represent the facts, which are either true or false. PL is not sufficient to represent the complex sentences or natural language statements. So we required some more powerful logic, such as first-order logic. First-order logic is also known as Predicate logic or First-order predicate logic. First-order logic is a powerful language that develops information about the objects in a more easy way and can also express the relationship between those objects. First-order logic (like natural language) does not only assume that the world contains facts like propositional logic but also assumes the following things in the world:

- **Objects**: A, B, people, numbers, colours, wars, theories, squares, pits, ...
- **Relations**: It can be unary relation such as: red, round, is adjacent, or n-any relation such as: the sister of, brother of, has colour, comes between
- **Function**: Father of, best friend, third inning of, end of, ...

Similar to natural language, first-order logic also has two main parts i.e. Syntax and Semantics.

Syntax of First-Order logic: The syntax of FOL determines which collection of symbols is a logical expression in first-order logic. The basic syntactic elements of first-order logic are symbols. We write statements in short-hand notation in FOL.

Basic Elements of First-order logic: The basic elements of FOL are shown in the table.

Constant	1, 2, A, John, Mumbai, Cat, ...
Variables	x, y, z, a, b,....
Predicates	Brother, Father, >,....
Function	sqrt, LeftLegOf, ...
Connectives	∧ , ∨ , ¬, ⇒ , ⇔
Equality	==
Quantifier	∀ , ☐

Atomic sentences:

- Atomic sentences are the most basic sentences of first-order logic. These sentences are formed from a predicate symbol followed by a parenthesis with a sequence of terms.
- We can represent atomic sentences as Predicate (term1, term2, ..., term n).

Example: Ravi and Ajay are brothers: Brothers (Ravi, Ajay).
Chinky is a cat: cat (Chinky).

Complex Sentences: Complex sentences are made by combining atomic sentences using connectives.

First-order logic statements can be divided into two parts:

- **Subject**: Subject is the main part of the statement.
- **Predicate**: A predicate can be defined as a relation, which binds two atoms together in a statement.

Consider the statement: "**x is an integer**." it consists of two parts, the first part **x is the subject** of the statement and second part "**is an integer" is known as a predicate.**

Quantifiers in First-order logic:

- A quantifier is a language element which generates quantification, and quantification specifies the quantity of specimen in the universe of discourse.
- These are the symbols that permit to determine or identify the range and scope of the variable in the logical expression. There are two types of quantifier:
  a. Universal Quantifier, (for all, everyone, everything)
  b. Existential quantifier, (for some, at least one).

Universal Quantifier: Universal quantifier is a symbol of logical representation, which specifies that the statement within its range is true for everything or every instance of a particular thing.
The Universal quantifier is represented by a symbol ∀ , which resembles an inverted A.

Note: In universal quantifier we use implication "→".
If x is a variable, then ∀x is read as: For all x, For each x, For every x

Example: ∀ x man(x) → drink (x, coffee).
It will be read as: There are all x where x is a man who drink coffee.

Existential Quantifier: Existential quantifiers are the type of quantifiers, which express that the statement within its scope is true for at least one instance of something.
It is denoted by the logical operator ☐ , which resembles as inverted E. When it is used with a predicate variable then it is called as an existential quantifier.

Note: **In Existential quantifier we always use AND or Conjunction symbol (∧).**
If x is a variable, then existential quantifier will be ☐x or ☐(x). And it will be read as:
There exists a 'x', For some 'x', For at least one 'x'

Example: ☐x: boys(x) ∧ intelligent(x)
It will be read as: There are some x where x is a boy who is intelligent.

Properties of Quantifiers:

- In universal quantifier, ∀x∀y is similar to ∀y∀x.
- In Existential quantifier, ∃x∃y is similar to ∃y∃x.
- ∃x∀y is not similar to ∀y∃x.

Some Examples of FOL using quantifier:

1. All birds fly.
In this question the predicate is "fly(bird)."
And since there are all birds who fly so it will be represented as follows.
∀ x bird(x) →fly(x).

2. Every man respects his parent.
In this question, the predicate is "respect(x, y)," where x=man, and y= parent.
Since there is every man so will use ∀ , and it will be represented as follows:
∀ x man(x) → respects (x, parent).

3. Some boys play cricket.
In this question, the predicate is "play(x, y)," where x= boys, and y= game. Since there are some boys so we will use ☐, and it will be represented as:
☐x boys(x) → play(x, cricket).

4. Not all students like both Mathematics and Science.
In this question, the predicate is "like(x, y)," where x= student, and y= subject.
Since there are not all students, so we will use ∀ with negation, so following representation for this:
¬∀ (x) [student(x) → like(x, Mathematics) ∧ like(x, Science)].

Inference in First-Order Logic: Inference in First-Order Logic is used to deduce new facts or sentences from existing sentences.

FOL inference rules for quantifier: As propositional logic we also have inference rules in first-order logic, so following are some basic inference rules in FOL:

- Universal Generalization
- Universal Instantiation
- Existential Instantiation
- Existential generalization

1. Universal Generalization:

- Universal generalization is a valid inference rule which states that if premise P(c) is true for any arbitrary element c in the universe of discourse, then we can have a conclusion as ∀ x P(x).
- It can be represented as: $\frac{P(c)}{\forall x\, P(x)}$
- This rule can be used if we want to show that every element has a similar property.
- In this rule, x must not appear as a free variable.

Example: Let's represent, P(c): "**A byte contains 8 bits**", so for **∀ x P(x),** "**All bytes contain 8 bits**." will also be true.

2. Universal Instantiation:

- Universal instantiation is also called as universal elimination is a valid inference rule. It can be applied multiple times to add new sentences.
- The new KB is logically equivalent to the previous KB.
- As per UI, **we can infer any sentence obtained by substituting a ground term for the variable**.
- The UI rule state that we can infer any sentence P(c) by substituting a ground term c (a constant within domain x) from **∀ x P(x) for any object in the universe of discourse**.
- It can be represented as: $\frac{\forall x\, P(x)}{P(c)}$

Example: 1. If "Every person like ice-cream"=> ∀x P(x). John is a person. So we can infer that "John likes ice-cream".

Example: 2. "All kings who are greedy are Evil." So let our knowledge base contains this detail as in the form of FOL: **∀x king(x) ∧ greedy (x) → Evil (x),**

So from this information, we can infer any of the following statements using Universal Instantiation:

- King(John) ∧ Greedy (John) → Evil (John),
- King(Richard) ∧ Greedy (Richard) → Evil (Richard),
- King(Father(John)) ∧ Greedy (Father(John)) → Evil (Father(John)),

3. Existential Instantiation:

- Existential instantiation is also called as Existential Elimination, which is a valid inference rule in first-order logic.
- It can be applied only once to replace the existential sentence.
- The new KB is not logically equivalent to old KB, but it will be satisfiable if old KB was satisfiable.
- This rule states that one can infer P(c) from the formula given in the form of ∃x P(x) for a new constant symbol c.
- The restriction with this rule is that c used in the rule must be a new term for which P(c) is true.
- It can be represented as: $\frac{\exists x\, P(x)}{P(c)}$

Example: From the given sentence: ∃ x Crown(x) ∧ OnHead(x, John), we can infer: Crown(K) ∧ OnHead(K, John), as long **as K does not appear in the knowledge base.**

The above used K is a constant symbol, which is called **Skolem constant.**

4. Existential introduction

- An existential introduction is also known as an existential generalization, which is a valid inference rule in first-order logic.
- This rule states that if there is some element c in the universe of discourse which has a property P, then we can infer that there exists something in the universe which has the property P.
- It can be represented as: $\frac{P(c)}{\exists x P(x)}$

Example: Let's say that, "Priyanka got good marks in English."
"Therefore, someone got good marks in English."

Resolution Principle: Resolution is a **theorem proving technique** which is based on **proofs by contradictions**.

Clause: Disjunction of literals (an atomic sentence) is called a **clause**. It is also known as a unit clause.

Conjunctive Normal Form: A sentence represented as a conjunction of clauses is said to be **conjunctive normal form** or **CNF**.

Resolution rule**:**

$$\frac{\alpha \vee \beta \quad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

Steps for Resolution:

1. Conversion of facts into FOL.
2. Convert FOL statements into CNF
3. Negate the desired conclusion (proof by contradiction)
4. Apply resolution rule and Draw resolution graph until either
   – Derive false (a contradiction)
   – Can't apply any more

Example:

(1) If it is sunny and warm day you will enjoy.
(2) If it is warm and pleasant day you will do strawberry picking
(3) If it is raining then no strawberry picking.
(4) If it is raining you will get wet.
(5) It is warm day
(6) It is raining
(7) It is sunny

Prove by resolution that:

(Goal - 1) You are not doing strawberry picking.
(Goal - 2) You will enjoy.

Step-1: Conversion of Facts into FOL:

(1) (sunny ∧ warm) → enjoy
(2) (warm ∧ pleasant) → strawberry_picking

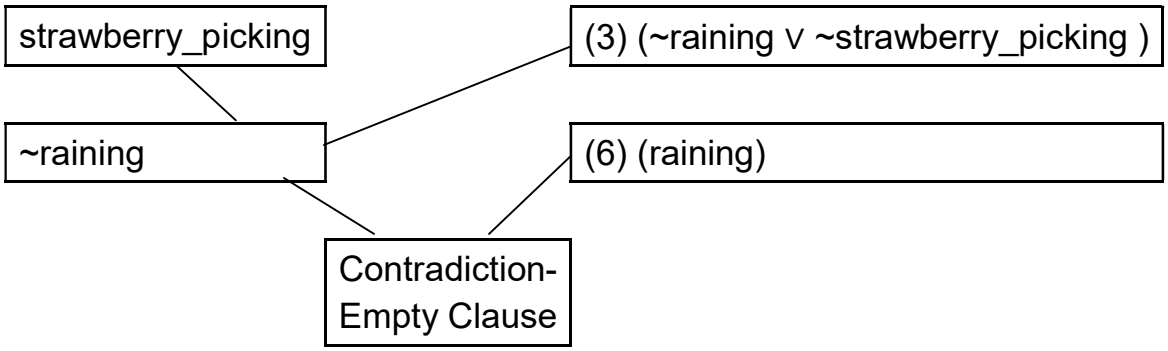
(3) raining → ~strawberry_picking
(4) raining → wet
(5) warm
(6) raining
(7) sunny

Step-2: Conversion of FOL into CNF:

(1) (~sunny ∨ ~warm ∨ enjoy)
(2) (~warm∨ ~pleasant ∨ strawberry_picking)
(3) (~raining ∨ ~strawberry_picking)
(4) (~raining ∨ wet)
(5) (warm)
(6) (raining)
(7) (sunny)

Goal 1 : You are not doing strawberry picking.

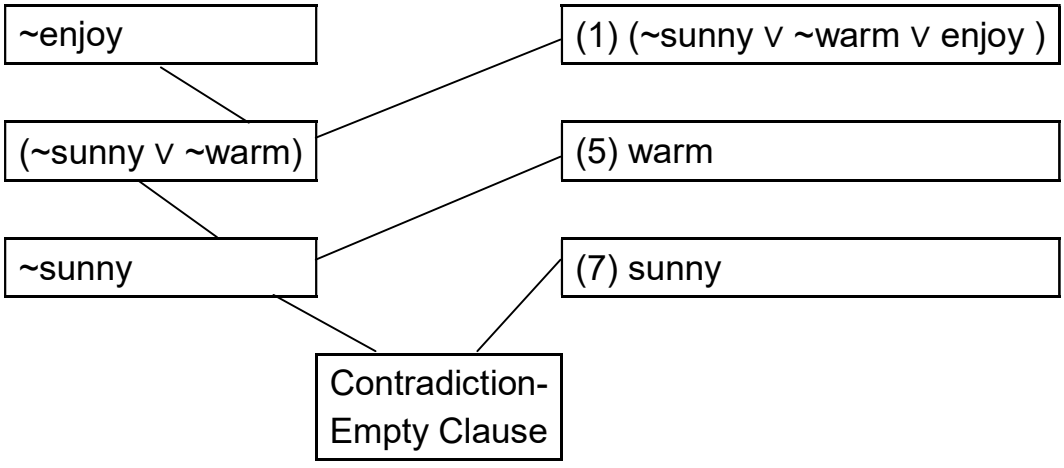
Prove : ~strawberry_picking
Assume : strawberry_picking (negate the goal and add it to given clauses).



Hence it is proved that "**You are not doing strawberry picking".**

Goal 2 : You will enjoy.

Prove : enjoy
Assume : ~enjoy (negate the goal and add it to given clauses)



Hence it is proved that "**You will enjoy**".

Unification: Unification is a process of making two different logical expressions identical by finding a substitution. The resulting substitution is called the **most general unifier (MGU).** It is used in automated reasoning technology, which remains one of the major application areas of unification.

Substitution: Substitution is a fundamental operation performed on terms and formulas. It occurs in all inference systems in first-order logic. The substitution is complex in the presence of quantifiers in FOL. If we write **F[a/x]**, it refers to substitute a constant "**a**" in place of variable "**x**". We may read it as: a for x.

Example: Find the MGU for the sentences **King(x)** and **King (John)**

Solution: Let Ψ_1 = King(x), Ψ_2 = King (John),
Substituting John in place of x we get, Ψ_1 = King (John)
So, Substitution **θ = {John/x}** is a unifier in this example because by applying this substitution, both expressions will be identical.

Conditions for Unification: Following are some basic conditions for unification:

- Predicate symbol must be same, atoms or expression with different predicate symbol can never be unified.
- Number of Arguments in both expressions must be identical.

1. Find the MGU of {p(f(a), g(Y)) and p(X, X)}

Sol: Here, Ψ_1 = p(f(a), g(Y)), and Ψ_2 = p(X, X)
Substitute **{f(a) / X}.** So, Ψ_1 = p(f(a), g(Y)), and **Ψ_2 = p(f(a), f(a))**
Substitute **{f(a) / g(y)}, Unification failed**.
Unification is not possible for these expressions.

2. Find the MGU of {p(b, X, f(g(Z))) and p(Z, f(Y), f(Y))}

Here, Ψ_1 = p(b, X, f(g(Z))) , and Ψ_2 = p(Z, f(Y), f(Y))
Substitute **{b/Z}.** Ψ_1 = p(b, X, f(g(b))); Ψ_2 = p(b, f(Y), f(Y))
Substitute **{f(Y) /X}** Ψ_1 = p(b, f(Y), f(g(b))); Ψ_2 = p(b, f(Y), f(Y))
Substitute **{g(b) /Y}** Ψ_1 = p(b, f(g(b)), f(g(b)); Ψ_2 = p(b, f(g(b)), f(g(b))
Unified Successfully And Unifier = {b/Z, f(Y) /X, g(b) /Y}.

3. Find the MGU of {p (X, X), and p (Z, f(Z))}

Here, Ψ_1 = {p (X, X), and Ψ_2 = p (Z, f(Z))
Substitute **{ Z / X }** Ψ_1 =p (Z, Z); Ψ_2 =p (Z, f(Z))
Substitute **{ Z / f(Z) }, Unification Failed**.
Hence, unification is not possible for these expressions.

4. Find the MGU of UNIFY(prime (11), prime(y))

Here, Ψ_1 = {prime(11) , and Ψ_2 = prime(y)
Substitute **{11/y}** Ψ_1 = prime(11) , Ψ_2 = prime(11)
Successfully unified and Unifier: {11/y}.

5. Find the MGU of Q(a, g(x, a), f(y)), Q(a, g(f(b), a), x)}

Here, Ψ_1 = Q(a, g(x, a), f(y)), and Ψ_2 = Q(a, g(f(b), a), x)
Substitute **{f(b)/x}** Ψ_1 = Q(a, g(f(b), a), f(y));
Ψ_2 = Q(a, g(f(b), a), f(b))
Substitute **{b/y}** Ψ_1 = Q(a, g(f(b), a), f(b));

Ψ_2 = Q(a, g(f(b), a), f(b)),

Successfully Unified. Unifier: [f(b)/x, b/y].

6. UNIFY(knows(Richard, x), knows(Richard, John))

Here, Ψ_1 = knows(Richard, x), and Ψ_2 = knows(Richard, John)

Substitute **{John/x}** Ψ_1 = knows(Richard, John);

Ψ_2 = knows(Richard, John)},

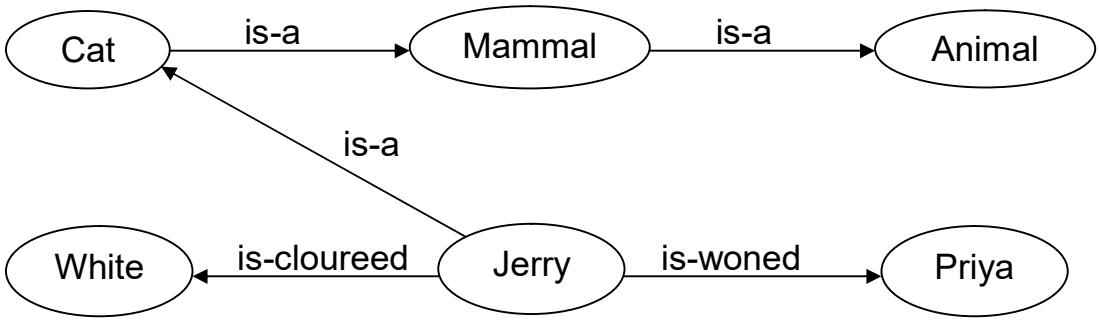
Successfully Unified. Unifier: {John/x}.

Semantic Nets: Semantic networks are alternative of predicate logic for knowledge representation. In Semantic networks, we can represent our knowledge in the form of graphical networks. This network consists of nodes representing objects and arcs which describe the relationship between those objects. Semantic networks can categorize the object in different forms and can also link those objects. Semantic networks are easy to understand and can be easily extended.

Example: Following are some statements which we need to represent in the form of nodes and arcs.

Statements:

a. Jerry is a cat.
b. Jerry is a mammal
c. Jerry is owned by Priya.
d. Jerry is white colored.
e. All Mammals are animal.



In the above diagram, we have represented the different type of knowledge in the form of nodes and arcs. Each object is connected with another object by some relation.

Drawbacks in Semantic representation:

1. Semantic networks take more computational time at runtime as we need to traverse the complete network tree to answer some questions. It might be possible in the worst case scenario that after traversing the entire tree, we find that the solution does not exist in this network.
2. Semantic networks try to model human-like memory (Which has 1015 neurons and links) to store the information, but in practice, it is not possible to build such a vast semantic network.
3. These types of representations are inadequate as they do not have any equivalent quantifier, e.g., for all, for some, none, etc.

4. Semantic networks do not have any standard definition for the link names.
5. These networks are not intelligent and depend on the creator of the system.

Advantages of Semantic network:

1. Semantic networks are a natural representation of knowledge.
2. Semantic networks convey meaning in a transparent manner.
3. These networks are simple and easily understandable.

Frames: A frame is a record like structure which consists of a collection of attributes and its values to describe an entity in the world. Frames are the AI data structure which divides knowledge into substructures. It consists of a collection of slots and slot values. These slots may be of any type and sizes. Slots have names and values which are called **facets**.

Facets: The various aspects of a slot are known as Facets. Facets are features of frames which enable us to put constraints on the frames. Example: IF-NEEDED facts are called when data of any particular slot is needed. A frame may consist of any number of slots, and a slot may include any number of facets and facets may have any number of values. A frame is also known as **slot-filter knowledge representation** in artificial intelligence.

A single frame is not much useful. Frames system consists of a collection of frames which are connected. In the frame, knowledge about an object or event can be stored together in the knowledge base. The frame is a type of technology which is widely used in various applications including Natural language processing and machine visions.

Example 1: Let's take an example of a frame for a book

Slots	Filters
Title	Artificial Intelligence
Genre	Computer Science
Author	Peter Norvig
Edition	Third Edition
Year	1996
Page	1152

Example 2: Let's suppose we are taking an entity, Peter. Peter is an engineer as a profession, and his age is 25, he lives in city London, and the country is England. So following is the frame representation for this:

Slots	Filter
Name	Peter
Profession	Doctor
Age	25
Marital status	Single
Weight	78

Advantages of frame representation:

1. The frame knowledge representation makes the programming easier by grouping the related data.
2. The frame representation is comparably flexible and used by many applications in AI.
3. It is very easy to add slots for new attribute and relations.
4. It is easy to include default data and to search for missing values.

5. Frame representation is easy to understand and visualize.

Disadvantages of frame representation:

1. In frame system, inference mechanism cannot be easily processed.

Production Rules: Production rules system consist of (**condition, action**) pairs which mean, "If condition then action". It has mainly three parts:

- The set of production rules
- Working Memory
- The recognize-act-cycle

In production rules agent checks for the condition and if the condition exists then production rule fires and corresponding action is carried out. The condition part of the rule determines which rule may be applied to a problem. And the action part carries out the associated problem-solving steps. This complete process is called a recognize-act cycle.

The working memory contains the description of the current state of problems-solving and rule can write knowledge to the working memory. This knowledge match and may fire other rules.

If there is a new situation (state) generates, then multiple production rules will be fired together, this is called conflict set. In this situation, the agent needs to select a rule from these sets, and it is called a conflict resolution.

Example:

- IF (at bus stop AND bus arrives) THEN action (get into the bus)
- IF (on the bus AND paid AND empty seat) THEN action (sit down).
- IF (on bus AND unpaid) THEN action (pay charges).
- IF (bus arrives at destination) THEN action (get down from the bus).

Advantages of Production rule:

1. The production rules are expressed in natural language.
2. The production rules are highly modular, so we can easily remove, add or modify an individual rule.

Disadvantages of Production rule:

1. Production rule system does not exhibit any learning capabilities, as it does not store the result of the problem for the future uses.
2. During the execution of the program, many rules may be active hence rule-based production systems are inefficient.

Scripts: A *script* is a structure that prescribes a set of circumstances which could be expected to follow on from one another. It is similar to a thought sequence or a chain of situations which could be anticipated. It could be considered to consist of a number of slots or frames but with more specialised roles.

The components of a script include:

Entry Conditions- These must be satisfied before events in the script can occur.

Results- Conditions that will be true after events in script occur.

Props- Slots representing objects involved in events.

Roles- Persons involved in the events.

Track (Variations on the script)- Different tracks may share components of the same script.

Scenes (The sequence of *events* that occur) *Events* are represented in *conceptual dependency* form.

Example: robbing a bank. This might involve:

- Getting a gun.
- Hold up a bank.
- Escape with the money.

Here the *Props* might be

- Gun, G.
- Loot, L.
- Bag, B
- Get away car, C.

The *Roles* might be:

- Robber, S.
- Cashier, M.
- Bank Manager, O.
- Policeman, P.

The *Entry Conditions* might be:

- S is poor.
- S is destitute.

The *Results* might be:

- S has more money.
- O is angry.
- M is in a state of shock.
- P is shot.

There are 3 scenes: obtaining the gun, robbing the bank and the getaway.

Advantages of Scripts:

- Ability to predict events.
- A single coherent interpretation may be build up from a collection of observations.

Disadvantages:

- Less general than frames.
- May not be suitable to represent all kinds of knowledge.